

Introduction to SWAR (SIMD Within A Register)

The Next Performance Frontier You Haven't Heard Of!

Jamie Pond

2024

What is SIMD?

What is SIMD?

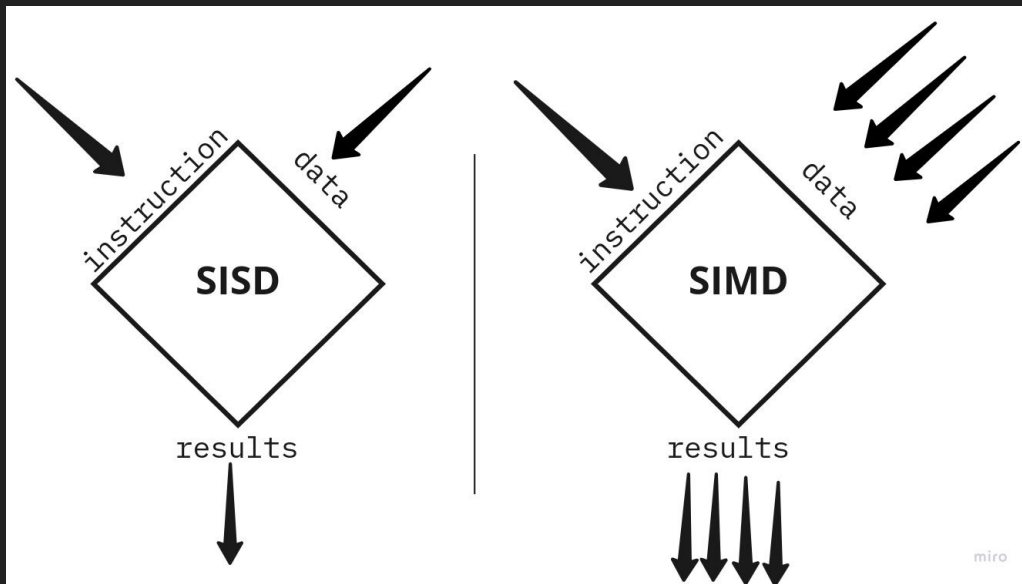
SIMD operations execute a Single Instruction against Multiple Data.

Data is said to be in multiple 'lanes'.

“Add lanes [1,2,3,4] to lanes [5,6,7,8], store in d for result [6,7,10,12]”

Does not *require* special hardware!

Scalar instructions execute against a single set of operands. Execute “1+5, store result 6 in c”



TLDW;

You don't need specialized hardware to benefit from SIMD.

TLDW;

SWAR is the cheapest parallelism there can be.

We need to think “wider” as developers

CUDA/HW-SIMD/SW-SIMD
AI is coming lol

By the end of this talk, you should...

- Be able to understand SWAR optimisations.
- Be able to implement some SWAR code to optimise functions for parallelisation.

Who am I?

- Lead Developer at mayk.it
- Music tech startup in Los Angeles
- I write a lot of different types of code
 - Lots of C++ in our audio engine
 - C++ in neural inference code
 - Some Swift for our iOS
 - Some Typescript
 - Plus a bunch more...
- MSc Computer Science from Queen Mary, University of London
- BA from the London College of Music.



Thanks!

Special thanks to **Eduardo Madrid** and **Scott Bruce**. Without whom this talk would not have been possible!

What is SWAR...?

```
constexpr u64 a = 0b0001; // 1
```

```
constexpr u64 b = 0b0010; // 2
```

```
constexpr u64 c = 0b0011; // 3
```

```
static_assert(a + b == c);
```

Just pretend that we have “SIMD” lanes!

```
constexpr u64 a = 0b00001'00001; // 1 | 1
constexpr u64 b = 0b0010'0010; // 2 | 2
constexpr u64 c = 0b0011'0011; // 3 | 3
static_assert(a + b == c);
```

Use any base type, to determine the upper limit of lanes!

```
constexpr u64 a = 0b0001'0001'0001'0001; // 1 | 1 | 1 | 1
constexpr u64 b = 0b0010'0010'0010'0010; // 2 | 2 | 2 | 2
constexpr u64 c = 0b0011'0011'0011'0011; // 3 | 3 | 3 | 3
static_assert(a + b == c);
```

This is the same as doing $(4369 + 8738 = 13'107)$

You just interpret the result differently!

Overflow! 🤪

```
constexpr u64 a = 0b0000'0001'0001'0001; // 0 | 1 | 1 | 1
constexpr u64 b = 0b0000'1111'0010'0010; // 0 | 15 | 2 | 2
constexpr u64 c = 0b0001'0000'0011'0011; // ! | ! | 3 | 3
static_assert(a + b == c);
```

Note: Library support and programmer diligence avoids over/underflow of lanes.

What does Software SIMD via SWAR cost?

- **Execution time cost is low:** we just use normal general purpose operations.
- **Software SIMD has no lane protection**, the cost comes at software implementation time.
- Ensuring correctness requires lots of bit manipulation, but these operations are very cheap!
- Superscalar CPUs do speculative, out of order, parallel instruction execution via multiple execution units.
- Adding 8 8-bit ints takes 1 add + 1 store and some bitwise ops: wildly faster than the normal 8 adds + 8 stores (16 vs 2 operations!)

Ops	Cost
Bitwise Ops, Shifts	~1
Bitwise Ops, Add, Subtract	~1
Multiply, Branch Hit, Predicted Indirect	1-3
Division, Modulo	~40
Branch Miss, Indirect Jump	10-60
L2 Cache Miss, Bad Indirect Jump	50-150

How do you use the library?

```
template<int NumBitsPerLane, typename UnderlyingType>
struct SWAR { /* impl */ };

static_assert([] {
    using S = swar::SWAR<8, uint64_t>; // 8 bit lanes, 64 bit underlying type
    S a{0x01'02'03'04'05'06'07'08};    // results in 8 lanes of 8 bits each
    S b{0x01'02'03'04'05'06'07'08};
    S c{0x02'04'06'08'0A'0C'0E'10};
    S sum = a + b;
    return horizontalEquality(sum, c);
})();
```

<https://github.com/thecppzoo/zoo>

Serial

```
// squares the elements of an array
void square(int* p, size_t n) {
    for (size_t i = 0; i < n; ++i)
        p[i] = p[i] * p[i];
}
```

ARM/Neon Intrinsic SIMD

```
void square_neon(int* p, size_t n) {  
    size_t i = 0;  
    for (; i + 3 < n; i += 4) {  
        // Load  
        int32x4_t vec = vld1q_s32(&p[i]);  
        // Square  
        vec = vmulq_s32(vec, vec);  
        // Restore  
        vst1q_s32(&p[i], vec);  
    }  
    // Process any remaining elements  
    for (; i < n; ++i)  
        p[i] = p[i] * p[i];  
}
```

SWAR Squaring!

```
template<typename S>
constexpr auto square_swar(S x) noexcept {
    return zoo::swar::multiplication_OverflowUnsafe(x, x);
}

using S = zoo::swar::SWAR<8, uint32_t>;
static_assert(square_swar(S{0x01'02'03'04}).value() == 0x01'04'09'10);
```

SWAR Equality?

[0101 | 0000 | 1000 | 0011] = $a \oplus b$, different lanes will be non-zero

[1000 | 1000 | 1000 | 1000] = `SWAR<4, uint16_t>::MostSignificantBit` (MSBs)

[1101 | 1000 | 1000 | 1011] = $(a \oplus b) \mid \text{MSBs}$, Ensures that the later sub1 is lane-safe.

[0001 | 0001 | 0001 | 0001] = `SWAR<4, uint16_t>::LeastSignificantBit` (LSBs)

[1??? | 0111 | 0??? | 1???] = $((a \oplus b) \mid \text{MSBs}) - \text{LSBs}$, subtract one identify MSBs on.

[0??? | 1000 | 1??? | 0???] = $\sim(((a \oplus b) \mid \text{MSBs}) - \text{LSBs})$

[0??? | 1000 | 0??? | 0???] = $\sim(((a \oplus b) \mid \text{MSBs}) - \text{LSBs}) \& (a \oplus b)$

[0000 | 1000 | 0000 | 0000] = `BooleanSWAR{~(((a^b) | MSBs) - xLSBs) & (a^b)}`

<https://godbolt.org/z/4c7Wxb1b6>

BooleanSWAR and PseudoBooleanSWAR

- Define the Most Significant Bit (MSB) of a lane as what contains its truth value. `||1000|0000|0000|1000||` has 2 true, 2 false lanes
- A PseudoBooleanSWAR can have at most 1 valid true in it, the least significant true present. Less significant lanes are false, more significant lanes are garbage. Ex: `||xxxx|xxxx|1000|0000||` has 2 dontcare lanes, 1 true lane, and 1 false lane.
- The BooleanSWAR technique allows converting control dependencies to data dependencies very efficiently!
 - Results in branchless computing with little overhead!
 - Search 8 bytes for a target in 7 cheap operations!
- PseudoBooleanSWAR (far as we can tell) has only been formalized as a concept in this library.

The SWAR Library is General Purpose!

SIMD with more freedom.

- Choose size of the lane.
- Create your own operations on the types. (Popcount, not available in SIMD!)
- Casting back to the underlying type and back is zero-cost.

We hope that eventually SWAR will become an implementation detail and we will think about this as a general purpose library for SIMD.

- Already stated on “Real”/hardware SIMD support
- GPU support in the future
- Combinations thereof!

Set operations are also available!

Set operations via SWAR already covered in Eduardo's pokerbot talk.

SWAR is a very general purpose technique, and this is not the only way you can use it.



The image shows a video player interface. On the left, a presentation slide features a cartoon character with curly hair and a blue suit, surrounded by colorful confetti. Below the character is a blue button with the text "Now we will see code!". To the right of the slide is a video player showing a man, Eduardo Madrid, speaking at a conference. The video player includes a title bar with "Cppcon 2019" and "The C++ Conference", a video thumbnail, and a title "Using Integers as Arrays of Bitfields a.k.a. SWAR Techniques - Eduardo Madrid - CppCon 2019". Below the title, it shows "3.5K views · 4 years ago" and the "CppCon" logo. The video description states: "SWAR techniques are described mostly in the HAKMEM memos and the 'Hacker's Delight' book. However, in the absence". At the bottom of the video player, there is a timeline with a red progress bar and a timestamp of "55:18". The video player also includes a list of topics: "OUTLINE | SWAR, Why bother? | Complete Application | SWAR AND C++ | Detecting a three of a kind |... 14 r".

Now we will see code!

Cppcon 2019
The C++ Conference
cppcon.org

Eduardo Madrid

How C++ Beats Other Languages at Card Games: Using Integers as Arrays of Bitfields, SWAR

Video Sponsorship Provided By
ansatz 55:18

Using Integers as Arrays of Bitfields a.k.a. SWAR Techniques - Eduardo Madrid - CppCon 2019
3.5K views · 4 years ago

CppCon

SWAR techniques are described mostly in the HAKMEM memos and the "Hacker's Delight" book. However, in the absence

OUTLINE | SWAR, Why bother? | Complete Application | SWAR AND C++ | Detecting a three of a kind |... 14 r

RobinHood Hash Table Implementation

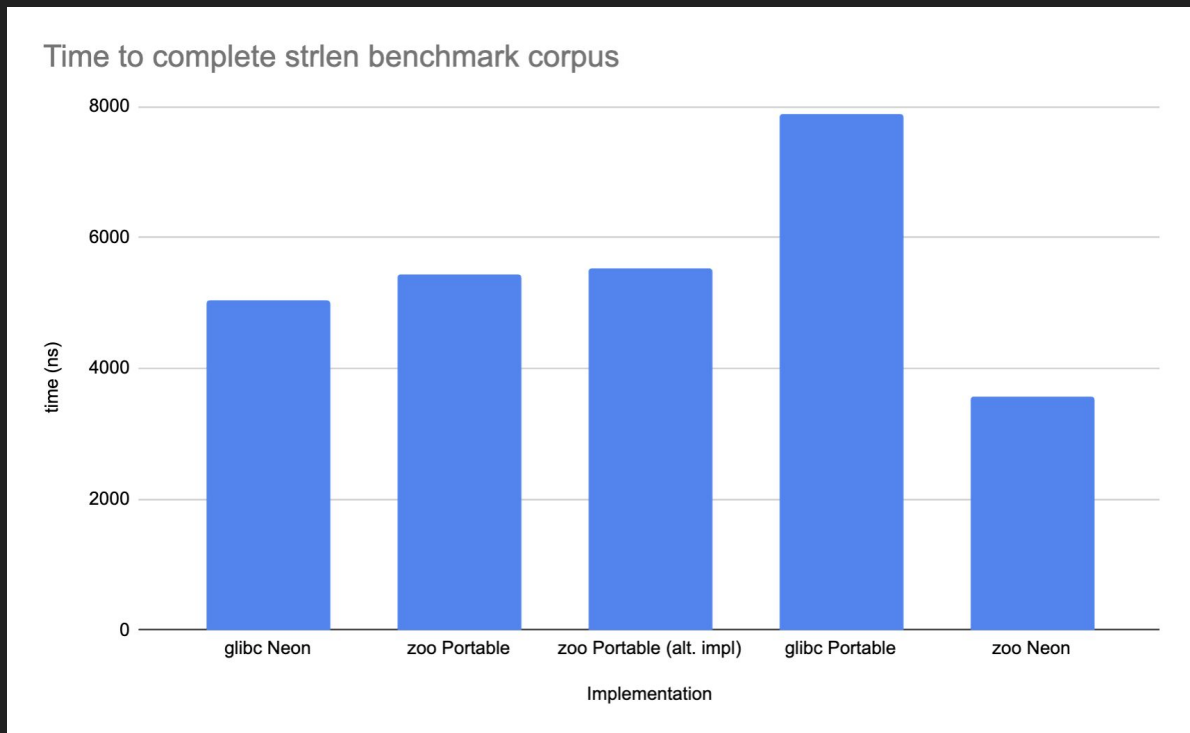


A Success Story in Implementing Understandable World Class Hash Tables

EDUARDO MADRID, SCOTT BRUCE



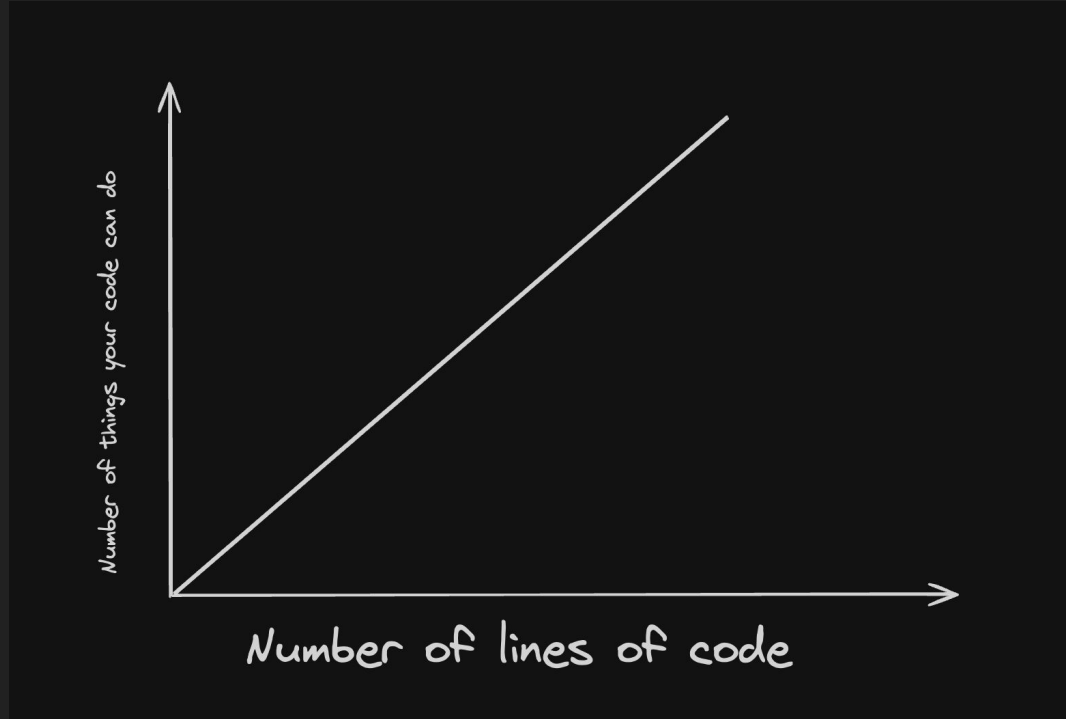
Some benchmarks...



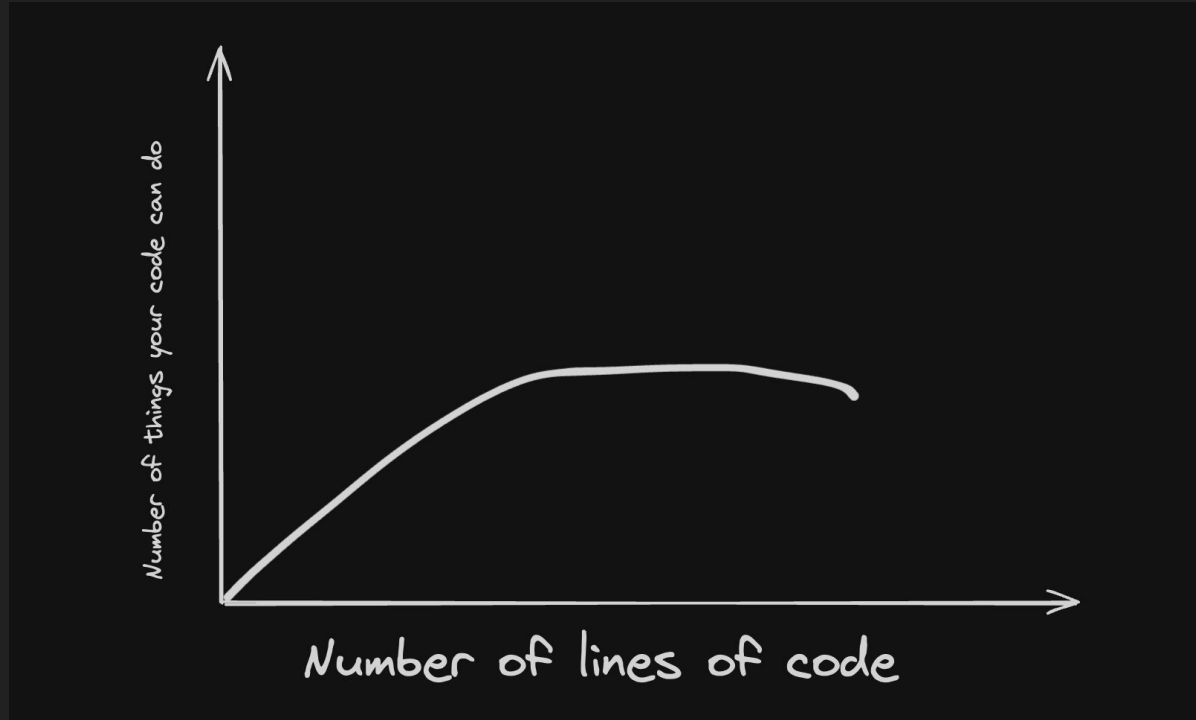
zoo Neon runs in ~70% the time of the glibc Neon (hand-crafted assembler).

Why is this so successful?

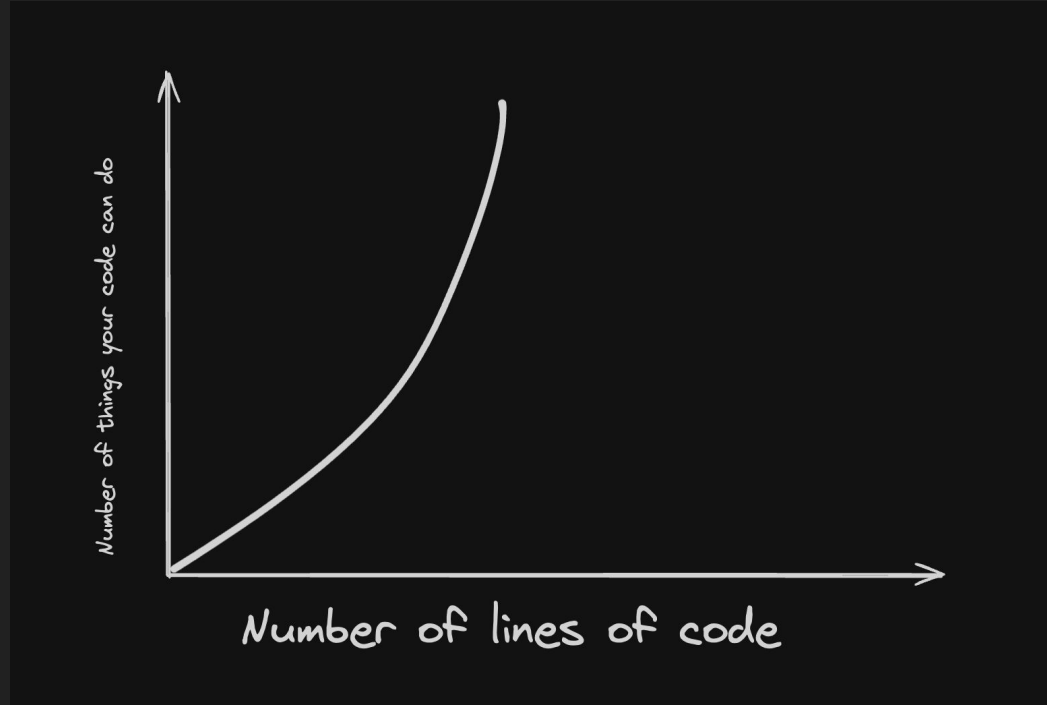
How people think software works...



How it works in reality...



A hopeful new way?



The Philosophy...

- *“Knowledge itself is power”*. - Sir Francis Bacon
- Knowledge composes (exponentially).
- If we are able to represent knowledge in source code, then we can have source code that grows in power exponentially. (As opposed to what we're used to).

Practical example -- Implementing SWAR `strlen`

```
static size_t strlen_simple(const char *s) {  
    size_t i = 0;  
    while (s[i] ≠ '\0') {  
        i++;  
    }  
    return i;  
}
```

How to do this in SWAR?

1. An aligned load!
 - a. We don't want to load invalid memory.
 - b. Aligned reads will guarantee that we're reading from the same page as the input.
 - c. The essence of misalignment is to avoid processing data that is not your input.
2. Figure out if any of the SWAR lanes are empty (which represents `\0`)
3. Return the index of `\0` within the SWAR if we found one.
4. Repeat until we found one!

More Godbolts!

<https://godbolt.org/z/9bjfrjKTE>

Interesting side note on GCC benchmarks...

```
static size_t naive(const char *s) {  
    size_t i = 0;  
    while (s[i]  $\neq$  '\0') {  
        i++;  
    }  
    return i;  
}
```

Interesting side note on GCC benchmarks...

```
naive(char const*):
```

```
    cmp     BYTE PTR [rdi], 0
```

```
    je      .L3
```

```
    sub     rsp, 8
```

```
    inc     rdi
```

```
    call    strlen
```

```
    add     rsp, 8
```

```
    inc     rax
```

```
    ret
```

```
.L3:
```

```
    xor     eax, eax
```

```
    ret
```

How to get started with the library using CMake

```
include(FetchContent)
FetchContent_Declare(
    zoo
    GIT_REPOSITORY https://github.com/thecppzoo/zoo.git
    GIT_TAG master # or some tag
    GIT_PROGRESS TRUE
    GIT_SHALLOW TRUE
)
FetchContent_Populate(zoo)
FetchContent_MakeAvailable(zoo)

# ...

target_include_directories(${PROJECT_NAME} PUBLIC ${zoo_SOURCE_DIR}/inc)
```

Summary / Wrap up

- SWAR is cool + fun.
- The library is co-evolving with it's applications
 - Pokerbotic
 - Robin Hood
 - String manipulations
- Represent knowledge in code for fun, powerful and exponentially returning results!
- Think wide™